

HANDS-ON PARALLEL PROGRAMMING

GPU Programming

A Practical Tutorial with CUDA C++

From a first kernel to shared-memory optimization

English edition • Local Linux or a cloud GPU notebook

What you will learn

Write a GPU kernel, map data to threads, transfer data between CPU and GPU, measure execution time correctly, and build parallel reduction and matrix multiplication kernels. Eight exercises guide the work, with source listings, commands, correctness checks, and investigation questions.

Audience and prerequisites

This tutorial is intended for students who know C or basic C++: arrays, functions, loops, pointers, and compilation. Prior MPI or OpenMP experience is useful but not required. The examples use a small subset of C++17, including `std::vector` for automatic host-memory management.

CUDA is the concrete programming platform used here. Running these examples requires access to a supported NVIDIA GPU and a compatible CUDA Toolkit. Students without such a GPU can use an available GPU runtime in Google Colab or a university GPU server. A CPU-only machine can be used to edit the code and study the algorithms, but it cannot execute these CUDA kernels.

Learning outcomes

After completing the tutorial, you should be able to:

1. Explain host/device execution and the grid–block–thread hierarchy.
2. Allocate device memory and move inputs and results explicitly.
3. Write bounds-safe kernels for one- and two-dimensional data.
4. Distinguish kernel time from transfer-inclusive application time.
5. Use block shared memory and synchronization without introducing races.
6. Validate every result and explain why an optimization helps or hurts.

Using this document

Upload this single `.tex` file to Overleaf and compile with pdfLaTeX. LaTeX only builds the document; execute the CUDA code in a GPU environment. Create the shared header once, then save each complete program using its listed filename. Listings explicitly labeled as replacements modify an earlier program. Copy code from the LaTeX source to avoid PDF line numbers.

Contents

1	Understand the CPU–GPU programming model	3
1.1	Host and device	3
1.2	Grid, block, thread, warp, and SM	3
1.3	The memory spaces used in this lab	4
1.4	The basic execution sequence	4
2	Prepare a GPU environment	4
2.1	Option A: Google Colab	4
2.2	Option B: A Linux workstation or GPU server	5
2.3	Compilation options	5
3	Create the shared error-checking header	6
3.1	Check both launch and execution	6
3.2	Files you will create	6
4	Exercise 1: Inspect the available GPU	7
5	Exercise 2: Launch a first kernel	8
6	Exercise 3: Add vectors using explicit device memory	9
7	Exercise 4: Measure kernel and transfer-inclusive time	11
8	Exercise 5: Tune launch size and memory access	13
9	Exercise 6: Reduce a vector with shared memory	14
10	Exercise 7: Map matrices to a two-dimensional grid	16
11	Exercise 8: Diagnose errors and explain performance	19
12	Optional mini-project: A two-dimensional stencil	20
13	Suggested report and evidence	20
14	Troubleshooting and quick reference	21
	References and further reading	22

1 Understand the CPU–GPU programming model

1.1 Host and device

The CPU runs the **host code**: preparing data, allocating memory, launching work, and checking results. The GPU runs **device code**. A function launched to execute across GPU threads is called a **kernel**. GPU acceleration is useful when enough independent work is available to amortize launch and data-movement costs.

CPU threads and GPU threads are software execution units, not interchangeable names for hardware cores. GPUs organize execution to sustain throughput across many threads. A GPU is not automatically faster for a small, serial, or transfer-dominated workload.

Aspect	OpenMP CPU example	CUDA example in this tutorial
Main execution	A CPU thread team	Host code launches a GPU grid
Data placement	Common process memory	Explicit host and device allocations
Work mapping	A loop schedule assigns iterations	Kernel code computes an index from block and thread IDs
Synchronization	Team barriers, reductions, locks	Block barriers, ordered kernel launches, and host/device synchronization

These are introductory usage patterns. CUDA also provides managed memory and more advanced execution mechanisms.

1.2 Grid, block, thread, warp, and SM

A kernel launch creates a **grid** of **thread blocks**. Each block contains threads identified by `threadIdx`; the block is identified by `blockIdx`. A **streaming multiprocessor (SM)** is a hardware execution unit that can host blocks. One block executes on one SM, and an SM may have multiple resident blocks. A large grid can execute in several waves [1].

A **warp** is a hardware scheduling group of 32 threads on current NVIDIA CUDA GPUs. Threads in a warp may follow different branches, but divergence can reduce efficiency. Do not assume that threads within a warp can safely exchange data without the synchronization required by the algorithm.

For a one-dimensional launch:

$$i = \underbrace{\text{blockIdx.x}}_{\text{block ID}} \underbrace{\text{blockDim.x}}_{\text{threads per block}} + \underbrace{\text{threadIdx.x}}_{\text{thread ID within block}} .$$

Block	Local thread IDs, for block size 8	Global indices
0	0 through 7	0 through 7
1	0 through 7	8 through 15
2	0 through 7	16 through 23

For 20 elements, the last four launched threads must avoid accessing the arrays.

1.3 The memory spaces used in this lab

Memory	Who uses it?	Practical role
Host memory	CPU code	Input generation, CPU reference, and returned results.
Device global memory	GPU threads across blocks	Large input/output arrays allocated with <code>cudaMalloc</code> .
Registers	An individual thread	Small per-thread values, subject to compiler allocation and register pressure.
Block shared memory	Threads in the same block	Explicit data reuse or communication within that block.

A normal pointer to device memory is not something host C++ code can dereference as an ordinary array. Copy device results to host memory before reading them. Conversely, the ordinary host buffers in this tutorial are copied to device allocations before the kernels access them.

A block barrier is not a grid barrier

`__syncthreads()` coordinates participating threads in one block. It does not wait for other blocks in an ordinary kernel. When a later phase depends on all blocks, a separate kernel in the same stream is a simple way to establish the phase boundary. All examples here use the default stream.

1.4 The basic execution sequence

1. Create inputs in host memory.
2. Allocate device buffers.
3. Copy input data from host to device (H2D).
4. Launch a kernel.
5. Ensure completion and copy results from device to host (D2H).
6. Validate the result and release device buffers.

A kernel launch normally returns control to the CPU before GPU execution has finished. A CPU timer placed only around the launch can therefore measure submission time rather than completed GPU work [3]. Exercise 4 introduces synchronized measurements.

2 Prepare a GPU environment

2.1 Option A: Google Colab

This route allows a student to use a browser even when the laptop has no NVIDIA GPU.

1. Open <https://colab.research.google.com/> and create a notebook.
2. In the runtime settings, select a GPU hardware accelerator and connect.
3. Run the following notebook cell to inspect the assigned environment.

```
!nvidia-smi
!nvcc --version
```

GPU models, access limits, and availability can change. A free GPU session is not guaranteed; use an available university GPU machine if no GPU is assigned. Colab runtime files are temporary, so save source files and the notebook outside the runtime before it is reset [10].

Create a working directory using two notebook cells:

```
!mkdir -p /content/cuda-lab
```

```
%cd /content/cuda-lab
```

Use the single-percent `%cd` notebook magic shown above. `!cd` runs in a temporary shell and does not change the directory of subsequent notebook cells.

To create a source file, start a cell with the following line and paste the entire corresponding listing below it:

```
%%writefile hello_gpu.cu
```

The `%%writefile` line belongs to the notebook cell, not to the C++ source. For the shared header, use `%%writefile cuda_check.cuh` instead. Compile and run with shell commands in later cells:

```
!nvcc -std=c++17 -O3 -lineinfo -arch=native hello_gpu.cu -o hello_gpu
!./hello_gpu
```

Use these commands after you have created the header and the source file. If `nvcc` is unavailable, select a CUDA development environment provided by your course or follow the official installation guide for the selected runtime; a working GPU driver alone is not a compiler installation.

2.2 Option B: A Linux workstation or GPU server

Use a supported NVIDIA GPU, an appropriate driver, and a compatible CUDA Toolkit. Follow NVIDIA's installation guide for the exact operating system and toolkit version rather than mixing unrelated driver packages [6].

```
nvidia-smi
nvcc --version
mkdir -p ~/cuda-lab
cd ~/cuda-lab
```

In a normal terminal, omit the leading `!` used in notebook cells. The CUDA Version field displayed by `nvidia-smi` describes driver compatibility; use `nvcc --version` to identify the compiler toolkit.

An ordinary desktop VM does not automatically expose its host's GPU for CUDA. Use a VM with supported GPU assignment, a GPU server, or a cloud GPU runtime. Windows users with supported NVIDIA hardware may also use the documented CUDA-on-WSL setup [8].

2.3 Compilation options

Option	Purpose
<code>-std=c++17</code>	Use the C++17 language standard.
<code>-O3</code>	Enable optimization; keep flags consistent in comparisons.
<code>-lineinfo</code>	Include device line information useful for diagnostics.
<code>-arch=native</code>	Target the supported GPU visible while compiling.

Recent toolkits support `-arch=native`. Compile on the machine where you will run the program, and recompile if your assigned GPU changes. For a different target, choose an explicit architecture supported by both the target GPU and your compiler [7]. An unsupported old GPU can require an older compatible toolkit.

3 Create the shared error-checking header

Create `cuda_check.cuh` in the same directory as all `.cu` files. This header makes every checked CUDA error visible and provides a numerical comparison that rejects non-finite results.

Listing 1: Save as `cuda_check.cuh`

```

1  #pragma once
2  #include <cuda_runtime.h>
3  #include <cstdio>
4  #include <cstdlib>
5  #include <cmath>
6
7  inline void cuda_check(cudaError_t error, const char *file, int line)
8  {
9      if (error != cudaSuccess) {
10         std::fprintf(stderr, "%s:%d: CUDA error: %s\n",
11                     file, line, cudaGetErrorString(error));
12         std::exit(EXIT_FAILURE);
13     }
14 }
15 #define CUDA_CHECK(call) cuda_check((call), __FILE__, __LINE__)
16
17 inline bool close_enough(double got, double expected,
18                           double atol = 1e-5, double rtol = 1e-5)
19 {
20     return std::isfinite(got) && std::isfinite(expected) &&
21         std::fabs(got - expected) <= atol + rtol * std::fabs(expected);
22 }

```

3.1 Check both launch and execution

After launching a kernel, introductory examples use:

Listing 2: Fragment following a kernel launch

```

1  CUDA_CHECK(cudaGetLastError());    // Check launch/configuration errors.
2  CUDA_CHECK(cudaDeviceSynchronize()); // Wait and check execution errors.

```

The first call does not prove that an asynchronous kernel completed correctly. The second waits for previously submitted work. In later timing examples, a synchronized event supplies the completion boundary. Production programs often synchronize more selectively to preserve concurrency.

Correctness is a separate question

A successful runtime call only means that the call did not report an error. It does not prove that a kernel computed the right values. Compare all outputs against a CPU reference or an independent mathematical result.

3.2 Files you will create

Filename	Role
<code>cuda_check.cuh</code>	Shared support header; create this first.
<code>device_info.cu</code>	Exercise 1: inspect the device.
<code>hello_gpu.cu</code>	Exercise 2: launch and identify threads.
<code>vector_add.cu</code>	Exercise 3: explicit memory and vector addition.
<code>vector_benchmark.cu</code>	Exercise 4: a copy of Exercise 3 with timed sections.
<code>reduce_sum.cu</code>	Exercise 6: reduction using block shared memory.
<code>matrix_mul.cu</code>	Exercise 7: matrix multiplication and a tiled variant.

Exercise 5 modifies the vector-add kernel; Exercise 8 applies diagnostics to the existing programs. Complete files and replacement fragments are identified explicitly below.

4 Exercise 1: Inspect the available GPU

Goal: record the actual hardware and device limits before selecting a launch configuration.

Listing 3: Save as `device_info.cu`

```

1 #include "cuda_check.cuh"
2
3 int main()
4 {
5     int count = 0;
6     CUDA_CHECK(cudaGetDeviceCount(&count));
7     if (count == 0) {
8         std::fprintf(stderr, "No CUDA device is available.\n");
9         return 1;
10    }
11    CUDA_CHECK(cudaSetDevice(0));
12    cudaDeviceProp p{};
13    CUDA_CHECK(cudaGetDeviceProperties(&p, 0));
14    std::printf("Device: %s\n", p.name);
15    std::printf("Compute capability: %d.%d\n", p.major, p.minor);
16    std::printf("SM count: %d\n", p.multiProcessorCount);
17    std::printf("Warp size: %d\n", p.warpSize);
18    std::printf("Maximum threads per block: %d\n", p.maxThreadsPerBlock);
19    std::printf("Global memory: %.2f GiB\n",
20                p.totalGlobalMem / (1024.0 * 1024.0 * 1024.0));
21    std::printf("Shared memory per block: %zu bytes\n",
22                static_cast<std::size_t>(p.sharedMemPerBlock));
23    return 0;
24 }
```

```

nvcc -std=c++17 -O3 -lineinfo -arch=native device_info.cu -o device_info
./device_info
```

In Colab, prefix each command with `!`. Output values depend on the assigned GPU; there is no universal model name or memory size to match.

Interpret the output

- **Compute capability** identifies an architecture feature level. It is different from the CUDA Toolkit version.
- **SM count** is not the maximum number of blocks in a grid. Blocks can be scheduled in waves, and multiple blocks may reside on an SM.
- **Maximum threads per block** is a limit, not a recommendation to always use the largest possible block.
- **Global memory** is device memory; it is distinct from host RAM.

Your tasks

1. Save the output along with `nvidia-smi` and `nvcc --version`.
2. Explain which reported limit constrains a 16-by-16-thread block.
3. Explain why a grid with more blocks than SMs is valid.

5 Exercise 2: Launch a first kernel

Goal: connect launch parameters to block and thread indices.

Listing 4: Save as `hello_gpu.cu`

```

1 #include "cuda_check.cuh"
2
3 __global__ void hello(int n)
4 {
5     int i = blockIdx.x * blockDim.x + threadIdx.x;
6     if (i < n) {
7         printf("block=%u local=%u global=%d\n",
8             blockIdx.x, threadIdx.x, i);
9     }
10 }
11
12 int main()
13 {
14     const int n = 20;
15     const int threads = 8;
16     const int blocks = (n + threads - 1) / threads;
17     hello<<<blocks, threads>>>(n);
18     CUDA_CHECK(cudaGetLastError());
19     CUDA_CHECK(cudaDeviceSynchronize());
20     std::printf("Host: the kernel has finished.\n");
21     return 0;
22 }
```

```

nvcc -std=c++17 -O3 -lineinfo -arch=native hello_gpu.cu -o hello_gpu
./hello_gpu
```

Expected behavior: 20 device greeting lines with global IDs 0 through 19, followed by the host message. The order of device greetings is unspecified. This very small launch is for observation, not a performance benchmark.

Understand the syntax

- `__global__` declares a kernel callable from the host and executed on the device.
- `hello<<<blocks, threads>>>(n)` supplies the number of blocks and threads per block, then passes `n` as a normal function argument.
- Integer ceiling division, `(n + threads - 1) / threads`, launches enough threads for these positive, small input sizes.
- `if (i < n)` guards surplus threads. Device `printf` is useful for small debugging examples, but can distort timing and has buffering limits.

Your tasks

1. Predict the number of launched threads and the number of active indices for `n=20`, `threads=8`. Check your prediction.
2. Repeat with `n=17` and `threads=16`.
3. Explain what changes if the bound check is removed in this print-only example, and why removing it from an array kernel can be a memory error.

6 Exercise 3: Add vectors using explicit device memory

Goal: implement the complete H2D–kernel–D2H workflow. Compute $c_i = a_i + b_i$. Each thread owns one output element.

Listing 5: Save as `vector_add.cu`

```

1  #include "cuda_check.cuh"
2  #include <vector>
3
4  __global__ void vector_add(const float *a, const float *b,
5                          float *c, int n)
6  {
7      int i = blockIdx.x * blockDim.x + threadIdx.x;
8      if (i < n) c[i] = a[i] + b[i];
9  }
10
11 int main()
12 {
13     const int n = 1000003;
14     const int threads = 256;
15     const int blocks = (n + threads - 1) / threads;
16     const std::size_t bytes = static_cast<std::size_t>(n) * sizeof(float);
17     std::vector<float> a(n), b(n), c(n);
18     for (int i = 0; i < n; ++i) {
19         a[i] = (i % 97) * 0.25f;
20         b[i] = (i % 31) * 0.5f;
21     }
22     float *d_a = nullptr, *d_b = nullptr, *d_c = nullptr;
23     CUDA_CHECK(cudaMalloc(reinterpret_cast<void **>(&d_a), bytes));
24     CUDA_CHECK(cudaMalloc(reinterpret_cast<void **>(&d_b), bytes));
25     CUDA_CHECK(cudaMalloc(reinterpret_cast<void **>(&d_c), bytes));
26
27     // GPU WORK: replace this block in Exercise 4.
28     CUDA_CHECK(cudaMemcpy(d_a, a.data(), bytes, cudaMemcpyHostToDevice));
29     CUDA_CHECK(cudaMemcpy(d_b, b.data(), bytes, cudaMemcpyHostToDevice));
30     vector_add<<<blocks, threads>>>>(d_a, d_b, d_c, n);
31     CUDA_CHECK(cudaGetLastError());
32     CUDA_CHECK(cudaDeviceSynchronize());
33     CUDA_CHECK(cudaMemcpy(c.data(), d_c, bytes, cudaMemcpyDeviceToHost));
34     // END GPU WORK
35
36     int errors = 0;
37     for (int i = 0; i < n; ++i) {
38         double expected = static_cast<double>(a[i]) + b[i];
39         if (!close_enough(c[i], expected)) ++errors;
40     }
41     std::printf("n=%d blocks=%d threads=%d errors=%d\n",
42               n, blocks, threads, errors);
43     std::printf("first=%.2f last=%.2f\n", c.front(), c.back());
44     CUDA_CHECK(cudaFree(d_a));
45     CUDA_CHECK(cudaFree(d_b));
46     CUDA_CHECK(cudaFree(d_c));
47     return errors != 0;
48 }

```

```

nvcc -std=c++17 -O3 -lineinfo -arch=native vector_add.cu -o vector_add
./vector_add

```

Success criterion: `errors=0`. With the supplied constants there are 3,907 blocks, and the last block includes threads beyond the array length. Those threads perform no memory access.

Read the memory operations

Operation	What it does
<code>std::vector<float> a(n)</code>	Allocates a host vector.
<code>cudaMalloc(..., bytes)</code>	Allocates a device buffer; its old contents are not initialized for you.
<code>cudaMemcpyHostToDevice</code>	Copies an input from the host vector to a device buffer.
<code>cudaMemcpyDeviceToHost</code>	Copies a computed device result to host memory.
<code>cudaFree(pointer)</code>	Releases a device allocation.

The output buffer does not need initialization because the kernel writes every valid output element before it is read. If a later kernel accumulates into an existing output, initialization becomes necessary.

Why this loop has no shared-write race

Thread i writes only $c[i]$, while all threads read inputs that remain unchanged. Sharing an array is safe here because output ownership is disjoint. This reasoning would fail if all threads updated the same scalar.

Your tasks

1. Test $n=1$, $n=255$, $n=256$, $n=257$, and the original $n=1000003$. Recompile after changing the constant.
2. Change the operation to $c_i = 2a_i + b_i$ and update the reference.
3. Explain the lifetime of each host and device allocation.
4. Explain why copying only the first output element would be an insufficient correctness test.

7 Exercise 4: Measure kernel and transfer-inclusive time

Goal: make two different performance questions explicit: how fast is the kernel when data is already on the GPU, and how long does a complete H2D–kernel–D2H operation take?

Make a copy of `vector_add.cu` named `vector_benchmark.cu`. Apply the following three edits to that copy. Keep the original unchanged.

Edit 1: Add a host clock

Add this header alongside the existing includes:

```
1 #include <chrono>
```

The CPU clock is needed for the serial baseline and transfer-inclusive time. CUDA events provide timestamps on the GPU timeline [5].

Edit 2: Add a serial reference and timer

Insert this block after initializing `a` and `b`, before declaring and allocating the device pointers:

Listing 6: Insert inside main after input initialization

```
1 using Clock = std::chrono::steady_clock;
2 std::vector<float> reference(n);
3 auto cpu_start = Clock::now();
4 for (int i = 0; i < n; ++i) {
5     reference[i] = a[i] + b[i];
6 }
7 double cpu_ms = std::chrono::duration<double, std::milli>(
8     Clock::now() - cpu_start).count();
```

In the existing validation loop, replace the computation of `expected` with:

```
1 double expected = reference[i];
```

This uses the timed CPU results for validation and prevents the reference calculation from becoming unused work that an optimizer could discard.

Edit 3: Replace the GPU WORK block

Replace everything between the `GPU WORK` and `END GPU WORK` comments with the next listing. The pointers have already been allocated. The first launch is an untimed warm-up; the timed operation uploads inputs again because it models one complete transfer-inclusive operation.

Listing 7: Replacement GPU WORK block

```
1 cudaEvent_t start, stop;
2 CUDA_CHECK(cudaEventCreate(&start));
3 CUDA_CHECK(cudaEventCreate(&stop));
4
5 // Warm up runtime and kernel before measuring.
6 CUDA_CHECK(cudaMemcpy(d_a, a.data(), bytes, cudaMemcpyHostToDevice));
7 CUDA_CHECK(cudaMemcpy(d_b, b.data(), bytes, cudaMemcpyHostToDevice));
8 vector_add<<<blocks, threads>>>(d_a, d_b, d_c, n);
9 CUDA_CHECK(cudaGetLastError());
10 CUDA_CHECK(cudaDeviceSynchronize());
11
12 auto wall_start = Clock::now();
13 CUDA_CHECK(cudaMemcpy(d_a, a.data(), bytes, cudaMemcpyHostToDevice));
14 CUDA_CHECK(cudaMemcpy(d_b, b.data(), bytes, cudaMemcpyHostToDevice));
15 CUDA_CHECK(cudaEventRecord(start));
16 vector_add<<<blocks, threads>>>(d_a, d_b, d_c, n);
17 CUDA_CHECK(cudaGetLastError());
18 CUDA_CHECK(cudaEventRecord(stop));
19 CUDA_CHECK(cudaEventSynchronize(stop));
```

```

20 CUDA_CHECK(cudaMemcpy(c.data(), d_c, bytes, cudaMemcpyDeviceToHost));
21 double operation_ms = std::chrono::duration<double, std::milli>(
22     Clock::now() - wall_start).count();
23
24 float kernel_ms = 0.0f;
25 CUDA_CHECK(cudaEventElapsedTime(&kernel_ms, start, stop));
26 std::printf("CPU=%.6f ms kernel=%.6f ms operation=%.6f ms\n",
27     cpu_ms, kernel_ms, operation_ms);
28 if (kernel_ms > 0.0f && operation_ms > 0.0) {
29     std::printf("resident-data speedup=%.3f operation speedup=%.3f\n",
30         cpu_ms / kernel_ms, cpu_ms / operation_ms);
31 }
32 CUDA_CHECK(cudaEventDestroy(start));
33 CUDA_CHECK(cudaEventDestroy(stop));

```

```

nvcc -std=c++17 -O3 -lineinfo -arch=native \
    vector_benchmark.cu -o vector_benchmark
./vector_benchmark

```

Keep the validation and all three `cudaFree` calls after the replacement. Never accept a timing result from a run that reports a CUDA error or fails the numerical check.

Interpret the measurements

Measurement	Included work
T_{CPU}	One serial host loop producing the reference vector.
T_{kernel}	GPU event interval around the kernel in the same stream; input/output transfers are excluded.
$T_{\text{operation}}$	H2D transfers, kernel launch/execution, synchronization, D2H transfer, and the associated host control overhead.

Allocation, context warm-up, input generation, validation, and printing are outside the reported intervals. Therefore $T_{\text{operation}}$ is the transfer-inclusive operation time, not the full executable's startup-to-exit time.

$$S_{\text{resident}} = \frac{T_{\text{CPU}}}{T_{\text{kernel}}}, \quad S_{\text{operation}} = \frac{T_{\text{CPU}}}{T_{\text{operation}}}.$$

All times in these ratios must use the same unit. It is possible for the resident-data speedup to exceed 1 while the operation speedup is below 1. Keeping data on the GPU across multiple operations can change that tradeoff.

Your tasks

1. Test $n = 1,003, 100,003, 1,000,003$, and $10,000,003$, subject to available memory.
2. Run each configuration at least five times. Keep raw measurements and report median times; calculate speedups from the corresponding medians.
3. Repeat very short kernels in a timed batch if individual event timings are unstable. Use identical inputs and divide the elapsed kernel interval by the number of launches; label this as a repeated, resident-data measurement.
4. Plot input size against both speedup definitions. Explain why the curves may differ.

n	CPU (ms)	Kernel (ms)	Operation (ms)	S_{resident}	$S_{\text{operation}}$
1,003					
100,003					
1,000,003					
10,000,003					

8 Exercise 5: Tune launch size and memory access

Goal: connect the mapping of threads to data with memory efficiency.

Start with adjacent elements

The vector kernel gives adjacent threads adjacent elements: thread 0 reads `a[0]`, thread 1 reads `a[1]`, and so on. For suitable alignment, accesses within a warp can be combined into efficient memory transactions, commonly called **coalesced accesses**. Scattered addresses can require more transactions for the same useful data [4].

Vector addition performs one addition while reading two `float` values and writing one. Its nominal useful traffic is $12n$ bytes. For a kernel time T_k in seconds, a useful effective-bandwidth estimate is

$$B_{\text{effective}} = \frac{12n}{T_k 10^9} \text{ GB/s.}$$

This is an algorithm-level traffic estimate, not a measurement of physical DRAM transactions. Caches, write behavior, and repeated inputs can affect the observed value.

Use a grid-stride loop

Make another copy of the working vector program. Replace only its `vector_add` kernel with the following implementation:

Listing 8: Replacement `vector_add` kernel

```

1  __global__ void vector_add(const float *a, const float *b,
2                          float *c, int n)
3  {
4      int first = blockIdx.x * blockDim.x + threadIdx.x;
5      int stride = blockDim.x * gridDim.x;
6      for (int i = first; i < n; i += stride) {
7          c[i] = a[i] + b[i];
8      }
9  }
```

Now set `blocks=128` and keep `threads=256` in the host code. The 32,768 launched threads cover the whole vector by processing additional elements separated by the grid's total thread count. Adjacent threads still access adjacent elements on each loop pass. The integer indexing is sufficient for the bounded sizes used in this lab; very large arrays require wider index arithmetic and launch-limit checks.

Your tasks

1. Verify that the grid-stride version produces the same result as the original mapping, including for a nonmultiple input size.
2. With a fixed input size, compare 64, 128, and 256 blocks, and 64, 128, and 256 threads per block. All must remain within the device limits.
3. Compare median kernel times and effective bandwidth. Do not assume the largest number of threads wins.
4. Explain why a single-thread launch can compute every element with the grid-stride kernel, but is unsuitable for high GPU throughput.

Occupancy is a resource measure, not a speed guarantee

Occupancy describes the active warps relative to the hardware maximum on an SM. Registers, shared memory, block size, and architectural limits influence it. More occupancy may help hide latency, but the fastest kernel need not have the highest occupancy. Measure the actual workload.

9 Exercise 6: Reduce a vector with shared memory

Goal: safely combine values from many threads.

In the following program, each block reduces one portion of an integer vector into a partial sum. The CPU then sums the partial results. This is a *GPU block reduction with a CPU final stage*; it is not a fully GPU-resident reduction.

All 256 threads first write one value into their block's shared array. Out-of-range threads write zero and still participate in the barriers. Each reduction step combines pairs of entries, halving the number of active additions.

Listing 9: Save as reduce_sum.cu

```

1  #include "cuda_check.cuh"
2  #include <vector>
3
4  constexpr int BLOCK = 256; // Must be a power of two for this algorithm.
5
6  __global__ void block_sum(const int *a, int *partial, int n)
7  {
8      __shared__ int values[BLOCK];
9      int t = threadIdx.x;
10     int i = blockIdx.x * blockDim.x + t;
11     values[t] = (i < n) ? a[i] : 0;
12     __syncthreads();
13
14     for (int offset = BLOCK / 2; offset > 0; offset /= 2) {
15         if (t < offset) values[t] += values[t + offset];
16         __syncthreads();
17     }
18     if (t == 0) partial[blockIdx.x] = values[0];
19 }
20
21 int main()
22 {
23     const int n = 1000003;
24     const int blocks = (n + BLOCK - 1) / BLOCK;
25     std::vector<int> a(n), partial(blocks);
26     long long expected = 0;
27     for (int i = 0; i < n; ++i) {
28         a[i] = i % 7;
29         expected += a[i];
30     }
31     int *d_a = nullptr, *d_partial = nullptr;
32     std::size_t bytes = static_cast<std::size_t>(n) * sizeof(int);
33     std::size_t partial_bytes =
34         static_cast<std::size_t>(blocks) * sizeof(int);
35     CUDA_CHECK(cudaMalloc(reinterpret_cast<void **>(&d_a), bytes));
36     CUDA_CHECK(cudaMalloc(reinterpret_cast<void **>(&d_partial),
37         partial_bytes));
38     CUDA_CHECK(cudaMemcpy(d_a, a.data(), bytes, cudaMemcpyHostToDevice));
39     block_sum<<<blocks, BLOCK>>>(d_a, d_partial, n);
40     CUDA_CHECK(cudaGetLastError());
41     CUDA_CHECK(cudaDeviceSynchronize());
42     CUDA_CHECK(cudaMemcpy(partial.data(), d_partial, partial_bytes,
43         cudaMemcpyDeviceToHost));
44     long long result = 0;
45     for (int x : partial) result += x;
46     std::printf("expected=%lld result=%lld match=%s\n",
47         expected, result, expected == result ? "yes" : "no");
48     CUDA_CHECK(cudaFree(d_a));
49     CUDA_CHECK(cudaFree(d_partial));
50     return result != expected;
51 }

```

```

nvcc -std=c++17 -O3 -lineinfo -arch=native reduce_sum.cu -o reduce_sum
./reduce_sum

```

Expected output for the default input:

```
expected=3000003 result=3000003 match=yes
```

Each input is between 0 and 6, so a 256-element block sum fits easily in an `int`. The CPU total uses `long long`. If you change data ranges, recheck overflow in every stage, not only in the final total.

Why the barriers are necessary

- The first barrier makes all initialized shared entries available before any thread reads its partner's value.
- Each loop barrier completes a reduction step before the next step reads its partial results.
- The conditional controls the addition, while the barrier is outside that condition so all block threads execute the same phase sequence.
- The kernel uses `BLOCK` both for storage and the reduction tree; launch it with exactly `BLOCK` threads.

Do not skip the shared-memory protocol

For this algorithm, an out-of-range thread must write a neutral zero and follow the block's synchronization sequence. An early return before that initialization would leave required entries undefined. Placing a barrier only in the `t < offset` branch also violates the intended collective phase protocol.

Atomics solve a different coordination problem

An unsynchronized global update such as `*total += a[i]` is a race when many threads update the same location. A compatible `atomicAdd` can make that single update atomic, but many contenders may serialize. A simple atomic variant must initialize the destination to zero before launching:

Listing 10: Fragments for an optional atomic experiment

```
1 // Host: d_total must point to a previously allocated int.
2 CUDA_CHECK(cudaMemset(d_total, 0, sizeof(int)));
3 // Device: after computing a valid index i.
4 atomicAdd(d_total, a[i]);
```

These are fragments, not a complete alternative program. An atomic operation does not create a grid-wide barrier or make unrelated memory accesses safe.

Your tasks

1. Test $n = 1, 255, 256, 257$ and the default size against the CPU reference.
2. Draw the additions for one block of eight values using a power-of-two version of the same algorithm.
3. Explain why choosing `BLOCK=192` breaks this halving algorithm even if the GPU supports a 192-thread block.
4. Optional: replace the CPU final stage with one or more additional GPU reduction launches using separate input/output buffers. Preserve correctness when the number of partial sums exceeds one block.

10 Exercise 7: Map matrices to a two-dimensional grid

Goal: implement matrix multiplication, then reuse input tiles in shared memory.

For $A \in \mathbb{R}^{M \times K}$ and $B \in \mathbb{R}^{K \times N}$, compute

$$C_{r,c} = \sum_{q=0}^{K-1} A_{r,q} B_{q,c}.$$

Store matrices in row-major order. Element $A_{r,q}$ is `a[r * k + q]`; element $C_{r,c}$ is `c[r * n + col]`. In the first kernel, each GPU thread computes one output element.

Listing 11: Save as `matrix_mul.cu`; complete program

```

1  #include "cuda_check.cuh"
2  #include <vector>
3
4  __global__ void matmul(const float *a, const float *b, float *c,
5                        int m, int k, int n)
6  {
7      int col = blockIdx.x * blockDim.x + threadIdx.x;
8      int row = blockIdx.y * blockDim.y + threadIdx.y;
9      if (row < m && col < n) {
10         float sum = 0.0f;
11         for (int q = 0; q < k; ++q) {
12             sum += a[row * k + q] * b[q * n + col];
13         }
14         c[row * n + col] = sum;
15     }
16 }
17
18 int main()
19 {
20     const int m = 127, k = 130, n = 129;
21     std::vector<float> a(m * k), b(k * n), c(m * n);
22     for (int i = 0; i < m * k; ++i) a[i] = ((i % 17) - 8) * 0.125f;
23     for (int i = 0; i < k * n; ++i) b[i] = ((i % 13) - 6) * 0.0625f;
24     std::size_t a_bytes = a.size() * sizeof(float);
25     std::size_t b_bytes = b.size() * sizeof(float);
26     std::size_t c_bytes = c.size() * sizeof(float);
27     float *d_a = nullptr, *d_b = nullptr, *d_c = nullptr;
28     CUDA_CHECK(cudaMalloc(reinterpret_cast<void *>(&d_a), a_bytes));
29     CUDA_CHECK(cudaMalloc(reinterpret_cast<void *>(&d_b), b_bytes));
30     CUDA_CHECK(cudaMalloc(reinterpret_cast<void *>(&d_c), c_bytes));
31     CUDA_CHECK(cudaMemcpy(d_a, a.data(), a_bytes, cudaMemcpyHostToDevice));
32     CUDA_CHECK(cudaMemcpy(d_b, b.data(), b_bytes, cudaMemcpyHostToDevice));
33     dim3 block(16, 16);
34     dim3 grid((n + block.x - 1) / block.x,
35              (m + block.y - 1) / block.y);
36     matmul<<<grid, block>>>(d_a, d_b, d_c, m, k, n);
37     CUDA_CHECK(cudaGetLastError());
38     CUDA_CHECK(cudaDeviceSynchronize());
39     CUDA_CHECK(cudaMemcpy(c.data(), d_c, c_bytes, cudaMemcpyDeviceToHost));
40     int errors = 0;
41     double max_error = 0.0;
42     for (int row = 0; row < m; ++row) {
43         for (int col = 0; col < n; ++col) {
44             double reference = 0.0;
45             for (int q = 0; q < k; ++q) {
46                 reference += static_cast<double>(a[row * k + q]) *
47                             b[q * n + col];
48             }
49             double got = c[row * n + col];
50             double error = std::fabs(got - reference);
51             if (error > max_error) max_error = error;
52             if (!close_enough(got, reference, 1e-4, 1e-4)) ++errors;
53         }
54     }
55     std::printf("M=%d K=%d N=%d errors=%d max_error=%.3e\n",
56               m, k, n, errors, max_error);
57     CUDA_CHECK(cudaFree(d_a));
58     CUDA_CHECK(cudaFree(d_b));
59     CUDA_CHECK(cudaFree(d_c));
60     return errors != 0;
61 }

```

```
nvcc -std=c++17 -O3 -lineinfo -arch=native matrix_mul.cu -o matrix_mul
./matrix_mul
```

Success criterion: `errors=0`. The deliberately unequal dimensions exercise partial blocks and make row/column indexing mistakes easier to detect. The numerical tolerance is a teaching choice for these bounded inputs, not a universal tolerance for matrix multiplication. A non-finite result is counted as an error even if the reported finite maximum error is small.

Use shared-memory tiles

Make a copy named `matrix_mul_tiled.cu`. Replace only the `matmul` kernel with the following definition and its `TILE` constant. Keep the `16,16` launch and all host validation code unchanged.

Listing 12: Replacement `matmul` kernel for `matrix_mul_tiled.cu`

```
1 constexpr int TILE = 16;
2
3 __global__ void matmul(const float *a, const float *b, float *c,
4                       int m, int k, int n)
5 {
6     __shared__ float a_tile[TILE][TILE];
7     __shared__ float b_tile[TILE][TILE];
8     int tx = threadIdx.x, ty = threadIdx.y;
9     int col = blockIdx.x * TILE + tx;
10    int row = blockIdx.y * TILE + ty;
11    float sum = 0.0f;
12
13    for (int base = 0; base < k; base += TILE) {
14        int a_col = base + tx;
15        int b_row = base + ty;
16        a_tile[ty][tx] = (row < m && a_col < k)
17            ? a[row * k + a_col] : 0.0f;
18        b_tile[ty][tx] = (b_row < k && col < n)
19            ? b[b_row * n + col] : 0.0f;
20        __syncthreads();
21        for (int q = 0; q < TILE; ++q) {
22            sum += a_tile[ty][q] * b_tile[q][tx];
23        }
24        __syncthreads();
25    }
26    if (row < m && col < n) c[row * n + col] = sum;
27 }
```

```
nvcc -std=c++17 -O3 -lineinfo -arch=native \
    matrix_mul_tiled.cu -o matrix_mul_tiled
./matrix_mul_tiled
```

Each block uses $2 \times 16 \times 16 \times 4 = 2,048$ bytes for these shared arrays. Threads cooperate to load tiles, reuse the values in the inner loop, and synchronize before overwriting them in the next iteration.

Boundary threads still help load the tile

Do not put an early bounds-based return before the tiled kernel's loop. A thread whose final output is out of range can still be responsible for a tile element needed by another thread. Zero padding handles invalid input positions while every thread follows the same synchronization sequence.

Your tasks

1. Verify both kernels using the supplied 127×130 and 130×129 inputs. Also test small dimensions below 16.
2. Test square dimensions 128, 256, and 512. Add CUDA event timing around the kernel, with warm-up outside the timed interval, as in Exercise 4.
3. Compare median kernel time for the naive and tiled versions. Use the same inputs, precision, compiler flags, and validation method.
4. Explain the role of each of the two barriers in a tile iteration.
5. Explain why a correct tiled implementation may still be slower on a small matrix. Treat an optimization as a hypothesis to measure.

For conventional dense multiplication, the approximate floating-point operation count is $2MKN$, counting one multiply and one add per inner iteration. Report a clearly labeled rate:

$$\text{GFLOP/s} = \frac{2MKN}{T_{\text{kernel}} 10^9}, \quad T_{\text{kernel}} \text{ measured in seconds.}$$

This teaching kernel does not use Tensor Cores or the full set of optimizations in vendor libraries. For application development, compare against a suitable library routine rather than assuming that a hand-written kernel is optimal.

11 Exercise 8: Diagnose errors and explain performance

Goal: use correctness tools and evidence to guide optimization.

Use Compute Sanitizer when available

Compile with `-lineinfo`, then run:

```
compute-sanitizer --tool memcheck ./vector_add
compute-sanitizer --tool memcheck ./reduce_sum
compute-sanitizer --tool racecheck ./reduce_sum
compute-sanitizer --tool synccheck ./reduce_sum
compute-sanitizer --tool memcheck ./matrix_mul_tiled
compute-sanitizer --tool racecheck ./matrix_mul_tiled
compute-sanitizer --tool synccheck ./matrix_mul_tiled
```

Tool	Main purpose in this lab
<code>memcheck</code>	Detect invalid memory accesses, including out-of-bounds accesses.
<code>racecheck</code>	Detect shared-memory access hazards. It is not a general proof that all global-memory accesses are race-free.
<code>synccheck</code>	Detect certain invalid synchronization usages.

Run memory checking before race checking. Numerical validation is still necessary: tools do not prove that an algorithm computes the intended result [9]. Some managed GPU environments do not provide these tools or the permissions they require. If unavailable, record that limitation and use an appropriate course GPU environment for this part.

Run controlled debugging experiments

Work on copies of the original programs so that you can restore the correct version.

1. In the vector-add copy, remove the `i < n` guard and use `n=257`, `threads=256`. Run `memcheck` and explain the invalid indices. Restore the guard.
2. In the reduction copy, remove the first barrier after writing `values[t]`. Run `racecheck` with several blocks and explain the possible read-before-write hazard. Restore the barrier.
3. Confirm that the restored programs pass their numerical checks. A bug may appear only intermittently; one correct-looking run is insufficient.

To help attribute asynchronous execution errors while debugging, a Linux terminal can run:

```
CUDA_LAUNCH_BLOCKING=1 ./vector_add
```

In Colab use `!env CUDA_LAUNCH_BLOCKING=1 ./vector_add`. This changes execution behavior. Remove it before collecting normal performance results, and do not use sanitizer runs as benchmark measurements.

Choose the next optimization from evidence

Observation	A useful next question
Kernel fast, operation slow	Can data stay on the GPU for several operations?
Small inputs slower on the GPU	Is there enough work to amortize launch and transfer costs?
More threads do not help	Is the bottleneck memory bandwidth, register pressure, or occupancy?
Tiling does not improve time	Is data reuse sufficient to repay loads and barriers?
Large timing variation	Did the GPU model, other activity, temperature, or clock state change?

12 Optional mini-project: A two-dimensional stencil

Implement a simple image filter or one simulation step on an $H \times W$ array. For each interior cell, compute the five-point average:

$$y_{r,c} = \frac{x_{r,c} + x_{r-1,c} + x_{r+1,c} + x_{r,c-1} + x_{r,c+1}}{5}.$$

Copy boundary cells unchanged. Use separate input and output arrays; an in-place update would create dependencies between neighboring threads.

1. Write a CPU reference and a GPU version with one thread per output cell.
2. Validate every output for irregular dimensions such as $H = 257$, $W = 259$, including all boundaries.
3. Add event timing and a transfer-inclusive measurement with clear scope.
4. Optionally add shared-memory tiles with a one-cell halo. Explain which thread loads each halo element and where synchronization is needed.
5. Run multiple simulation steps while retaining data on the GPU, swapping input and output buffers between ordered launches. Compare against repeated H2D and D2H transfers, while computing exactly the same number of steps.

This mini-project connects GPU programming to image processing and numerical simulation. It also makes memory reuse and transfer costs visible without requiring a large external dataset.

13 Suggested report and evidence

1. **Environment:** GPU name, compute capability, driver/toolkit versions, execution platform, and compiler flags.
2. **Correctness:** source code, commands, outputs, and edge-size tests for every completed exercise.
3. **Performance:** raw repeated measurements, median times, launch configurations, and clearly defined timing boundaries.
4. **Plots:** vector size versus kernel/operation speedup; matrix size versus naive/tiled kernel time.
5. **Analysis:** explain at least one slowdown, one improvement, and one bug you diagnosed.
6. **Reproduction:** include the shared header and all modified sources, or link a repository containing them.

Report observed results, including negative speedups. Do not substitute expected performance for measurements, and do not combine results from different GPUs without identifying the hardware difference.

14 Troubleshooting and quick reference

Symptom	What to inspect
<code>nvcc: command not found</code>	Verify that a CUDA Toolkit is installed and its compiler is on the path. A driver installation alone is insufficient.
No CUDA device available	Check GPU runtime selection or server allocation. A CPU-only VM cannot execute these kernels.
<code>cuda_check.cuh</code> not found	Save the shared header in the same working directory as the program, or add an appropriate compiler include path.
Unsupported architecture or no kernel image	Verify GPU/toolkit compatibility. Recompile on the assigned GPU with a supported architecture target.
Invalid configuration argument	Check block dimensions, total threads per block, and shared-memory requirements against device limits.
Illegal memory access	Check bounds, element counts versus byte counts, pointer ownership, and matrix index arithmetic.
An error appears at a later CUDA call	Kernels are asynchronous. Check launch errors and use an appropriate completion boundary to isolate execution errors.
Correctness depends on size or launch	Test partial blocks and tiles; inspect shared writes, missing initialization, and synchronization.
Tiny or implausible time	Check synchronization, units, timer placement, and whether the timed output is actually used and validated.

Construct or API	Meaning
<code>__global__</code>	A kernel executed on the GPU.
<code>kernel<<<grid, block>>>(...)</code>	Launch work with a specified hierarchy.
<code>threadIdx, blockIdx</code>	The current thread and block coordinates.
<code>blockDim, gridDim</code>	The launch dimensions.
<code>cudaMalloc / cudaFree</code>	Allocate or release device memory.
<code>cudaMemcpy</code>	Transfer data using the specified direction.
<code>cudaGetLastError</code>	Query/reset the thread's last runtime error status.
<code>cudaDeviceSynchronize</code>	Wait for prior device work and report errors.
<code>__shared__</code>	Declare block-local shared storage.
<code>__syncthreads()</code>	A block-level synchronization primitive.
<code>cudaEventElapsedTime</code>	Read the interval between recorded events in ms.

References

- [1] NVIDIA. *CUDA Programming Guide: Programming Model*. <https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/programming-model.html>.
- [2] NVIDIA. *CUDA Programming Guide: Intro to CUDA C++*. <https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/intro-to-cuda-cpp.html>.
- [3] NVIDIA. *CUDA Programming Guide: Asynchronous Execution*. <https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/asynchronous-execution.html>.
- [4] NVIDIA. *CUDA Programming Guide: Writing SIMT Kernels*. <https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/writing-cuda-kernels.html>.
- [5] NVIDIA. *CUDA C++ Best Practices Guide*, especially GPU timing, memory traffic, and performance measurement. <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>.
- [6] NVIDIA. *CUDA Installation Guide for Linux*. <https://docs.nvidia.com/cuda/cuda-installation-guide-linux/>.
- [7] NVIDIA. *CUDA Compiler Driver NVCC*. <https://docs.nvidia.com/cuda/cuda-compiler-driver-nvcc/index.html>.
- [8] NVIDIA. *CUDA on WSL User Guide*. <https://docs.nvidia.com/cuda/wsl-user-guide/index.html>.
- [9] NVIDIA. *Compute Sanitizer Documentation*. <https://docs.nvidia.com/compute-sanitizer/ComputeSanitizer/index.html>.
- [10] Google. *Colab Frequently Asked Questions*. <https://research.google.com/colaboratory/faq.html>.

The programs are teaching examples intended for a compatible NVIDIA GPU environment. They illustrate explicit memory management and conventional CUDA kernels. Hardware performance must be measured on the actual assigned device. The exact platform, toolkit, and available tools should be recorded with each set of experimental results.